

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS			
	NOMBRE	CARGO	FECHA	Código: GTI-OT016
Elaborado por:	Mauricio Benites	Analista de Procesos y Mejora Continua	08/05/2025	
Revisado por:	José Salazar	Jefe de Arquitectura y Base de datos	26/05/2025	Versión: 01
Aprobado por:	José Salazar	Jefe de Arquitectura y Base de datos	26/05/2025	Página: 1 de 26

1. OBJETIVO:

Establecer los lineamientos y estándares necesarios para la creación y gestión de bases de datos relacionales y NoSQL para el desarrollo de productos, asegurando un enfoque ordenado, estructurado y comprensible para todos los involucrados. Este documento tiene como propósito definir las prácticas que garanticen la consistencia, integridad, rendimiento y seguridad de los datos.

2. ALCANCE:

Este documento tiene como alcance desde la nomenclatura hasta los lineamientos de seguridad, rendimiento y buenas prácticas para garantizar la integridad y eficiencia de los datos.

3. TERMINOLOGÍA:

- 3.1. **Base de Datos Relacional:** Sistema que almacena datos estructurados en tablas con relaciones definidas mediante llaves primarias y foráneas.
- 3.2. **Base de Datos NoSQL:** Sistema de gestión de datos que no sigue el modelo relacional tradicional. Ideal para datos distribuidos, no estructurados o semiestructurados.
- 3.3. **Clave de Partición (Partition Key):** Atributo principal que determina cómo se distribuyen los datos en una base de datos NoSQL.
- 3.4. **Clave de Ordenación (Sort Key):** Atributo secundario que permite ordenar y segmentar los datos dentro de una misma partición en NoSQL.
- 3.5. **Constraint:** Restricción que se aplica a los datos de una tabla para garantizar integridad y validez.
- 3.6. **Foreign Key (FK):** Campo que crea una relación entre dos tablas, apuntando a la llave primaria de otra tabla.
- 3.7. **Índice (Index):** Estructura que mejora el rendimiento de las consultas al permitir accesos más rápidos a los datos.
- 3.8. **Job:** Proceso automatizado que ejecuta tareas específicas dentro del servidor de base de datos, como cargas o transformaciones.
- 3.9. **Nomenclatura:** Convención estandarizada para nombrar objetos de base de datos, facilitando su identificación y mantenimiento.
- 3.10. **Primary Key (PK):** Campo o conjunto de campos que identifican de forma única cada registro en una tabla.
- 3.11. **Soft Delete:** Técnica que evita la eliminación física de un registro, marcándolo como inactivo mediante un campo lógico.
- 3.12. **Stored Procedure:** Conjunto de instrucciones SQL almacenadas en la base de datos que se ejecutan como una sola unidad lógica.
- 3.13. **Transacción:** Conjunto de operaciones que se ejecutan como una unidad atómica, garantizando consistencia en la base de datos.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 2 de 26

3.14. Trigger: Objeto de base de datos que se ejecuta automáticamente ante eventos DML, DDL o LOGON.

3.15. Vista (View): Objeto de base de datos que presenta una representación lógica de datos a partir de una o más tablas.

4. DOCUMENTOS DE REFERENCIA:

No aplica.

5. LINEAMIENTOS

- 5.1 Es responsabilidad del Líder difundir y/o comunicar el presente procedimiento a todos los colaboradores involucrados, así como de velar por el cumplimiento del mismo y dictar las disposiciones de detalle para alcanzar la finalidad del presente.
- 5.2 Ante cualquier cambio del documento producto de alguna observación por auditoría, normativa legal y/o mejora de procesos, es responsabilidad del Usuario Líder, solicitar de manera formal mediante correo a los Líderes del área de Procesos y Mejora Continua la actualización del mismo.
- 5.3 El presente documento tiene vigencia por 4 años desde la fecha de su publicación.
- 5.4 Las bases de datos son el núcleo de los sistemas y aplicativos modernos, permitiendo el almacenamiento, recuperación y análisis de grandes volúmenes de información. En el contexto de CORPORACIÓN TURÍSTICA PERUANA SAC (CTP), es fundamental contar con lineamientos claros para la creación, gestión y mantenimiento de objetos de bases de datos, tanto relacionales como NoSQL, para asegurar consistencia, integridad, rendimiento y seguridad de los datos.

LINEAMIENTOS PARA BASES DE DATOS RELACIONALES

- 5.5 Una base de datos relacional es un sistema de almacenamiento y gestión de datos que organiza la información en tablas con filas y columnas, donde cada tabla representa una entidad o concepto del negocio. Los datos en estas tablas están relacionados entre sí mediante llaves primarias (Primary Keys) y llaves foráneas (Foreign Keys), lo que permite establecer conexiones lógicas entre diferentes conjuntos de datos.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 3 de 26

5.6 Para la creación de objetos, se deben seguir las siguientes reglas:

#	GRUPO	REGLA
1	Evitar usar NCHAR, NVARCHAR, NTEXT	Solo aplicarlo para casos específicos , como cuando se desea almacenar información como el idioma chino y japonés.
2	No dejar líneas obsoletas o comentadas	No dejar líneas obsoletas (líneas comentadas) dentro del cuerpo de un procedimiento almacenado, funciones, vistas u otros objetos de programación de base de datos.
3	Nombres de objetos de base de datos	Usar minúsculas para los nombres de objetos como tablas, vistas, procedimientos, funciones, etc. Esto es por consistencia y evitar confusión entre nombres similares que se escriban en mayúsculas o minúsculas.
		No usar espacios en los nombres de los objetos. Si es necesario usar nombres de varias palabras, utiliza el guion bajo (_) para separarlas.
		Evitar que los nombres empiecen con un guion bajo (_), ya que puede ser confuso y menos legible.
		Ser consistente con los nombres. Por ejemplo, si decides usar id_customer como nombre para la clave de la tabla Customer, debes mantener este mismo patrón en todo el esquema de la base de datos.
4	Evitar columnas NULLABLES	Evitar columnas NULLABLES , es decir, columnas que pueden tener valores nulos. Esto puede indicar que tu base de datos no está normalizada correctamente, lo que puede generar problemas de consistencia en los datos.
		La normalización es un proceso que organiza los datos para reducir la redundancia. Sin embargo, hay que encontrar un equilibrio correspondiente a los requerimientos de cada proyecto en este punto. Como regla general la tercera forma normal (3FN) es un buen punto intermedio.
5	Uso de DISTINCT y SELECT*	Evitar el uso de DISTINCT ya que puede afectar el rendimiento de la base de datos. Esto ocurre porque DISTINCT obliga al motor de la base de datos a procesar los datos de manera adicional para eliminar duplicados.
		Evitar el uso de SELECT*. Es preferible especificar las columnas que realmente necesitas en la consulta para mejorar el rendimiento.
6	Uso de ORDER BY, GROUP BY y funciones en la cláusula WHERE	Evitar el uso excesivo de ORDER BY en consultas. Se sugiere realizar la ordenación en el lado del cliente si es posible.
		Utilizar GROUP BY con funciones de agregación para resumir los datos de manera eficiente.
		Evitar en la cláusula WHERE , el uso de funciones numéricas, fechas y cadena, cuando compara una columna con una constante, ya que esto afecta el rendimiento en el motor de base de datos. Por ejemplo, no hacer "Select campo1, campo2 from tabla where WHERE YEAR(campo2)=2024".
7	Uso del TOP para limitar registros	Cuando se desea traer un conjunto específico de registros , usar la cláusula TOP para limitar la cantidad de filas que se devuelven.
8	Transacciones y manejo de errores	Cuando trabajas con transacciones explícitas, siempre debes iniciar con BEGIN TRANSACTION y asegurarte de terminar con COMMIT o ROLLBACK dependiendo de si la operación fue exitosa o no.
		Usar el bloque TRY.CATCH para manejar errores en procedimientos almacenados y verificar el contador de transacciones (@@TRANCOUNT) antes de hacer un ROLLBACK.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 4 de 26

#	GRUPO	REGLA
9	Valores por defecto	Usar valores por defecto sobre las columnas donde el aplicativo o proceso de carga no grabara un valor. Esto es importante para mantener la consistencia de los datos y alinear con las reglas del negocio.
10	Uso de IDENTITY vs Secuencia	No usar IDENTITY para generar claves primarias. En su lugar, utilizar Secuencias, estas son más flexibles y escalables, especialmente a partir de SQL Server 2012.
11	Llaves Primarias y Foráneas	Toda tabla debe tener una llave primaria (PK) para garantizar que cada registro sea único.
		Se sugiere que los campos que conforman la llave primaria (PK) deben ser aquellos que son utilizados por las consultas más frecuentes, ya que dichos campos están dentro de un índice tipo clúster.
		Definir llaves foráneas (FK) para asegurar la integridad referencial entre las tablas, evitando que se inserten datos huérfanos o inconsistentes.
12	Índices	Usar índices CLÚSTER y NO CLÚSTER. El índice CLÚSTER organiza físicamente los datos en la tabla, lo que lo hace más rápido para la lectura, pero sólo puede haber uno por tabla.
		El índice NO CLÚSTER se utiliza para acelerar las consultas, pero puede haber varios de estos por tabla.
		Evitar tener tablas sin índices CLÚSTER y NO CLÚSTER, más aún si estas tablas participan en un reporte y consultas de alto volumen de información.
13	Carácter comodín (%)	Evitar usar el comodín % al inicio de un patrón en el operador LIKE en la cláusula WHERE, ya que esto genera un impacto negativo en el rendimiento.
14	Coincidencia de tipos de datos en variables y parámetros	Los tipos de datos usados para crear variables o parámetros en procedimientos, funciones y otros objetos de programación deben coincidir con los tipos de datos de las columnas de la tabla que serán utilizadas.
15	Tablas sin datos innecesarios	Evitar tablas u otros objetos innecesarios en la base de datos, ya que esto genera un mantenimiento adicional y puede afectar el rendimiento.
16	Partición de tablas grandes	Las tablas grandes deben ser particionadas para mejorar el rendimiento de las consultas. La partición puede ser por fecha (por ejemplo, particionar por año o mes), o por conjunto de valores.
17	Tablas históricas	Crear tablas históricas con la finalidad de que en las tablas transaccionales o principales deba existir la información necesaria para las consultas del aplicativo. Esto ayudará mucho en los tiempos de respuestas de las consultas.
18	Soft Delete	El proceso de eliminación de una información a través de un sistema o aplicativo (API, servicio, microservicio u otros) no debe ser una eliminación física del registro (Soft Delete), sino una actualización del registro para que no se visualice en el sistema o aplicativo.
19	Módulos de Mantenimiento, Configuración y Auditoría	Todo nuevo aplicativo o sistema debe tener un módulo de Mantenimiento, Configuración y Auditoría de DML (lenguaje de manipulación de datos - UPDATE, INSERT, DELETE), con el objetivo de evitar que los cambios sobre las tablas, se hagan a través de script con sentencia DML.
20	No usar Servidores Vinculados	Evitar el uso de servidores vinculados (Linked Server), ya que pueden generar problemas de rendimiento y seguridad. Es preferible utilizar otras técnicas para compartir datos entre servidores.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 5 de 26

5.7 Para la nomenclatura de objetos, se deben seguir los siguientes lineamientos:

- La longitud máxima permitida para los nombres de bases de datos y esquemas será de 30 caracteres.
- Los nombres de las bases de datos correspondientes a los sistemas informáticos de Producción o Desarrollo deberán reflejar de manera clara el Proceso de Negocio que respaldan. En el caso de las bases de datos temporales, el nombre de la base restaurada deberá incluir la fecha en la que se realizó la copia de seguridad, utilizando el formato YYYYMMDD (Año, Mes, Día).
- Una vez creada la base de datos para un sistema informático, y en caso de ser necesario establecer una base de datos adicional vinculada al mismo sistema, se deberá crear un esquema dentro de la base de datos principal.

Ejemplo: En el caso de la empresa Atlantic City, que cuenta con la Dirección de Recursos Humanos, y considerando que esta dirección posee tres (03) oficinas, a saber: Bienestar, Planilla y Marcación, la nomenclatura de la base de datos y sus esquemas será la siguiente: La base de datos principal se denominará **bd_recursoshumanos**. Los esquemas correspondientes a cada oficina se denominarán de la siguiente manera: **sch_bienestar** para la oficina de Bienestar, **sch_planilla** para la oficina de Planilla y **sch_marcacion** para la oficina de Marcación.

5.8 Para la nomenclatura de una **base de datos**, se deben seguir las siguientes reglas:

- Se deberá anteponer el prefijo identificador de dos (02) caracteres "**bd**", con el fin de indicar que se trata de una base de datos, seguido del nombre correspondiente al Proceso de Negocio. En el caso del entorno de desarrollo, se añadirá al final el sufijo "**dev**" para distinguirlo claramente.

TIPO	SINTAXIS	EJEMPLOS
Producción o QC	bd_<nombre proceso>	bd_promociones
Desarrollo	bd_<nombre proceso>_dev	bd_promociones_dev
Temporal	bd_<nombre proceso>_<yyyymmdd>	bd_promociones_20241106

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 6 de 26

5.9 Para la nomenclatura de un **esquema**, se deben seguir las siguientes reglas:

- Se deberá anteponer el prefijo identificador de tres (03) caracteres "**sch**", con el fin de indicar que se trata de un esquema, seguido del nombre correspondiente al Subproceso de Negocio. Además, se debe evitar el uso del esquema por defecto DBO.

TIPO	SINTAXIS	EJEMPLOS
Producción o QC	sch_<nombre sub_proceso>	sch_bienestar
Desarrollo	sch_<nombre sub_proceso>	sch_bienestar
Temporal	sch_<nombre sub_proceso>	sch_bienestar

5.10 Para la nomenclatura de una **tabla**, se deben seguir las siguientes reglas:

- El nombre de una tabla deberá ser lo más descriptivo posible y utilizar el singular, procurando incluir las características más relevantes de la información que contendrá.
- El nombre de la tabla no debe exceder los 30 caracteres, debe estar libre de tildes y espacios en blanco, y, en el caso de nombres compuestos, estos deben estar separados por guión bajo (_).
- En el caso de que se utilicen campos compuestos, solo la última palabra significativa deberá ir en plural.
- Se deberá anteponer el prefijo identificador de un (01) carácter: "T", para indicar que se trata de un objeto de tipo tabla.
- En el caso de las tablas, a este prefijo le seguirá un identificador que indique el tipo de tabla, conforme a la nomenclatura definida.

TIPO	IDENTIFICADOR
Maestro	M
Transacción	T
Histórico	H
Auditoría	A
Temporal	X

- Para las **tablas de auditoría**, después de colocar el prefijo identificador de tipo de tabla, se añadirá el nombre de la instancia a la cual pertenece la data auditada. Además, tras ingresar el prefijo y el identificador, se deberá incluir una abreviatura que indique el propósito de la tabla (por ejemplo, **invt** para invitados, **jueg** para juegos), seguida del nombre correspondiente de la tabla.

Ejemplos:

- tm_invt_invitado:** Tabla maestra de invitados.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 7 de 26

- **tt_opro_operacion:** Tabla transaccional de las operaciones de los invitados.
 - **tt_movt_movimiento:** Tabla transaccional de los movimientos de los invitados.
 - **th_opro_operacion:** Tabla histórica de las operaciones de los invitados.
 - **ta_proddb07_bd_promocion:** Tabla de auditoría DML sobre la base de datos promoción, perteneciente a la instancia **PRODDb07**. Dicha tabla, por razones de seguridad, se aloja en otra instancia.
 - **tx_grp_grupo_jugadores:** Tabla temporal de los grupos de invitados.
- Para una base de datos tipo **Datamart** o **Data Warehouse**, las tablas con el rol de dimensión deberán tener el prefijo **dim**, seguido del nombre correspondiente. Por otro lado, las tablas con el rol de hechos deberán tener el prefijo **fact**, seguido del nombre correspondiente.

Ejemplos:

- **dim_articulo:** Tabla de dimensión de artículos.
- **dim_cartera:** Tabla de dimensión de cartera.
- **fact_juegos:** Tabla de hechos de juegos.
- **fact_pedido:** Tabla de hechos de pedidos.

5.11 Para la nomenclatura de un **campo**, se deben seguir las siguientes reglas:

- Los nombres de los campos deben ser singulares, simples, representativos e intuitivos. En caso de utilizar nombres compuestos, las palabras deben separarse por guion bajo (_). Además, la longitud máxima permitida para los nombres de los campos es de 30 caracteres. Si el nombre del campo es compuesto, únicamente la última palabra significativa debe ir en plural.
- Los campos que forman parte de una llave primaria (PK) deberán ser los primeros en la tabla.
- En los casos en que se utilicen varias tablas en una consulta y se presenten campos ambiguos, se deberá utilizar un alias sobre las tablas para evitar confusiones.
- Si un campo o columna forma parte de una clave foránea (Foreign Key), el nombre del campo deberá coincidir con el nombre de la tabla de origen con la que se relaciona.
- La sintaxis para el nombre de los campos seguirá el formato previamente establecido, con las normas aquí indicadas.
- Cada tabla deberá incluir un campo que indique el estado del registro. Este campo se utilizará para evitar la eliminación física del registro, implementando un Soft Delete. El nombre de este campo será **"fl_persistente"** y su tipo de dato será **"bit"** (1=verdadero, 0=falso), lo que indica si el registro es persistente o no.

- La sintaxis del nombre del campo deberá tener el siguiente formato:

```

1 <AbrevTabla><Abrevtype><Nombrecampo>
2
3 <AbrevTabla> = Abreviatura de la tabla, la cual pertenece al campo.
4
5 <Abrevtype> = Abreviatura que indica el Tipo de datos.
6
7 char = c
8
9 varchar = v
10
11 nvarchar = nv
12
13 datetime = dt
14
15 <Nombrecampo> = Descripción clara que identifica el campo.

```

Ejemplo:

- inv_t_c_nombre:** Representa el campo “nombre” dentro de la tabla de invitados (inv_t).
 - inv_t_dt_fechanacimiento:** Representa el campo “fecha de nacimiento” dentro de la tabla de invitados (inv_t).
- Todas las tablas deben incluir un total de siete (07) campos de auditoría, que son necesarios para el seguimiento y control de la información en la base de datos.

CAMPOS	DESCRIPCIÓN	TIPO DE DATOS
usuario_registro	Usuario que creó el registro.	varchar(30)
fecha_registro	Fecha en que se creó el registro.	datetime
usuario_modificacion	Usuario que modificó el registro.	varchar(30)
fecha_modificacion	Fecha que se modifica el registro.	datetime
ip	Registra la IP del usuario que hizo el cambio.	varchar(30)
flg_persistente	Flag de eliminación lógica del registro.	bit
hostname	Registra el nombre del host del usuario que hizo el cambio.	varchar(50)

5.12 Para la nomenclatura de una **vista**, se deben seguir las siguientes reglas:

- El nombre de una vista debe ser lo más descriptivo posible, procurando incluir las características más relevantes de la información que contendrá.
- El nombre de la vista no debe contener espacios en blanco. En el caso de nombres compuestos, se deberá utilizar el guion bajo (_) para separar las palabras.
- La longitud máxima permitida para los nombres de las vistas es de 30 caracteres.
- Se deberá anteponer el prefijo identificador de tres (03) caracteres: “**uvw**”, para indicar que se trata de una vista.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 9 de 26

SINTAXIS	EJEMPLO
uvw_<nombre_vista>	uvw_jugadores_fisicos

5.13 Para la nomenclatura de una **función**, se deben seguir las siguientes reglas:

- El nombre de una función debe ser lo más descriptivo posible, procurando reflejar las características más relevantes de la información que procesará o devolverá.
- El nombre de la función no debe contener espacios en blanco. En el caso de nombres compuestos, se deberá utilizar el guion bajo (_) para separar las palabras.
- La longitud máxima permitida para los nombres de las funciones es de 30 caracteres.
- Se deberá anteponer el prefijo identificador de tres (03) caracteres según el tipo de función:
 - "ufn" para indicar que se trata de una función de valor escalar.
 - "tfn" para indicar que se trata de una función valor tabla.

TIPO	SINTAXIS	EJEMPLO
Valor Escalar	ufn_<nombre_funcion>	ufn_fechadiacasino_
Valor Tabla	tfn_<nombre_funcion>	tfn_autoexclusion

5.14 Para la nomenclatura de **procedimiento almacenados**, se deben seguir las siguientes reglas:

- El nombre de un procedimiento almacenado debe ser lo más descriptivo posible, reflejando las características más relevantes de la información o acción que ejecutará.
- El nombre de un procedimiento no debe contener espacios en blanco. En el caso de nombres compuestos, se debe utilizar el guion bajo (_) para separar las palabras.
- La longitud máxima permitida para los nombres de los procedimientos almacenados es de 30 caracteres.
- Se deberá anteponer el prefijo identificador de tres (03) caracteres: "**usp**", para indicar que se trata de un procedimiento almacenado de usuario. Es importante evitar el uso del prefijo "**sp**", ya que esto podría hacer que el motor de base de datos busque primero en el catálogo del sistema (master), en lugar de en el catálogo de usuario.

SINTAXIS	EJEMPLO
usp_<nombre_procedimiento>	usp_obtener_invitado_ganadores_torneo_calimaco
usp_<nombre_procedimiento>	usp_obtener_jugadores_lobbie

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 10 de 26

5.15 Para la nomenclatura de **secuencias**, se deben seguir las siguientes reglas:

- El nombre de una secuencia debe estar relacionado directamente con el nombre de la tabla a la cual se asociará. Por ejemplo, si la tabla se llama **tt_opro_operaciones**, la secuencia correspondiente se denominará **useq_operaciones**.
- El nombre de la secuencia no debe contener espacios en blanco. En el caso de nombres compuestos, se deberá utilizar el guion bajo () para separar las palabras.
- La longitud máxima permitida para los nombres de las secuencias es de 30 caracteres.
- Se deberá anteponer el prefijo identificador de cuatro (04) caracteres: "**useq**", para indicar que se trata de una secuencia de usuario.

SINTAXIS	EJEMPLO
useq_<nombre_secuencia>	useq_operaciones
usp_<nombre_secuencia>	useq_movimientos

5.16 Para la nomenclatura de **tipo de datos** y **tipo de tabla** definido por el usuario, se deben seguir las siguientes reglas:

- El nombre de un tipo de datos debe ser lo más descriptivo posible, reflejando las características más relevantes de la información que contendrá.
- El nombre de un tipo de datos no debe contener espacios en blanco. En el caso de nombres compuestos, se debe utilizar el guión bajo () para separar las palabras.
- La longitud máxima permitida para los nombres de los tipos de datos es de 30 caracteres.
- Se deberá anteponer el prefijo identificador de cuatro (04) caracteres:
 - "**uddt**" para indicar que se trata de un **tipo de datos** definido por un usuario
 - "**udtt**" para indicar que se trata de un **tipo de tabla** definido por un usuario

TIPO	SINTAXIS	EJEMPLO
Valor Dato	<uddt>_<nombre_tipodatos>	uddt_nuevoalias
Valor Tabla	<udtt>_< nombre_tipotabla>	Udtt_grillaparametro

5.17 Para la nomenclatura de **triggers**, se deben seguir las siguientes reglas:

- El nombre de un trigger debe ser lo más descriptivo posible, reflejando las características más relevantes de la información o acción que ejecutará.
- El nombre del trigger no debe contener espacios en blanco. En el caso de nombres compuestos, se deberá utilizar el guión bajo () para separar las palabras.
- La longitud máxima permitida para los nombres de los triggers es de 30 caracteres.
- Se deberá anteponer el prefijo identificador de tres (03) caracteres: "**TRG**", para indicar que se trata de un trigger.
- Los triggers se clasifican en tres (03) tipos:
 - DML (Data Manipulation Language)

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 11 de 26

- DDL (Data Definition Language)
- LOGON (Para eventos de inicio de sesión)
- Para los triggers de tipo DML, se deberá añadir un identificador en la parte final de su nombre que refleje el tipo de acción que realiza (por ejemplo: INSERT, UPDATE, DELETE)

IDENTIFICADOR	DESCRIPCIÓN
UPD	Acción de Actualización
INS	Acción de Inserción
DEL	Acción de Eliminación
ALL	Realiza UPD y INS

- **IMPORTANTE:** El trigger que valida el evento de eliminación se aplicará únicamente cuando el usuario realice la eliminación del registro de manera directa en la base de datos. Es decir, no deberá activarse si la eliminación se lleva a cabo a través del aplicativo o sistema.

TIPO	SINTAXIS	EJEMPLO
DML	<trg>_<dml>_<tabla>_<identificador>	trg_dml_tt_eventoregistro_upd
		trg_dml_tt_eventoregistro_ins
DDL	<trg>_<ddl>_<nombreevento>	trg_ddl_capturacambios
LOGON	<trg>_<log>_<nombreevento>	trg_log_listablanca

5.18 Para la nomenclatura de **usuarios de base de datos**, se deben seguir las siguientes reglas:

- El nombre del usuario de base de datos debe ser lo más descriptivo posible, reflejando las características relacionadas con el proceso de negocio que soporta.
- La longitud máxima permitida para los nombres de los usuarios es de 45 caracteres.
- Se deberá anteponer el prefijo identificador de tres (03) caracteres: "APP", para indicar que se trata de un usuario de base de datos asociado a una aplicación o carga de trabajo.
- Después de añadir el prefijo APP, se debe incluir un identificador que indique el tipo de tecnología o herramienta utilizada.



**LINEAMIENTOS DE DEFINICION DE ESTANDARES Y
BASE DE DATOS**

Código: GTI-OT016

Versión: 01

Página: 12 de 26

TIPO	DESCRIPCIÓN
ETL	Usuario que será utilizado en proyectos de Extracción, Transformación y Carga de Datos.
WEB	Usuario que será utilizado para aplicaciones WEB.
API	Usuario que será utilizado para una aplicación tipo API.
WS	Usuario que será utilizado por un servicio WEB.
DSKT	Usuario que será utilizado para una aplicación tipo escritorio.
MS	Usuario que será utilizado por un microservicio.
DATAPROC	Usuario que será utilizado para proyectos de Ingesta de datos en Nube GCP.
LS	Usuario que será utilizado por un Link Server, para aplicaciones Legacy.
PB	Usuario que será utilizado por Power BI.
LMDA	Usuario que será utilizado por el servicio AWS Lambda.
OTROS	Para otros propósitos

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 13 de 26

TIPO	SINTAXIS	EJEMPLO
ETL	app_etl_hostOrigen_hostDest_r_proceso	App_etl_nodo1_p01_r_SorteoCalimaco
	app_etl_hostOrigen_hostDest_r_proceso	trg_ddl_capturacambios
	app_etl_hostOrigen_hostDest_rw_proceso	trg_log_listablanca
API	app_api_hostOrigen_r_proceso	app_api_p07_r_sorteoestelar
	app_api_hostOrigen_rw_proceso	app_api_p07_rw_sorteoestelar
WEB	app_web_hostOrigen_proceso	app_web_p03_crmcloud
WS	app_ws_hostOrigen_proceso	app_ws_p02_calculopuntos
DSKT	app_dskt_hostOrigen_proceso	app_dskt_2k8_notarios
MS	app_ms_hostOrigen_proceso	app_ms_p07_generaopciones
DATAPROC	app_dataproc_hostOrigen_hostDest_r_proceso	app_dataproc_stag_gcp_r_gastosprom
LS	app_lsl_hostOrigen_hostDest_proceso	app_ls_p01_p07_calculocuentas

5.19 Para la nomenclatura de **tipo de constraints**., se deben seguir la siguiente regla:

- La longitud máxima permitida para los constraints es de 30 caracteres.

Los constraints permiten aplicar reglas a nivel de tabla para controlar y validar los datos. Por otro lado, contribuyen a garantizar la integridad de los datos dentro de la base de datos y evitan la eliminación de registros en tablas hijas o en la tabla maestra cuando existen relaciones de dependencia.

TIPO	SINTAXIS	EJEMPLO
Primary Key	pk_nomtabla	pk_tt_invexcl_invitadosexcluidos
Foreign Key	fk_tablahijo_tablapadre	fk_tm_ditto_distrito_tt_invexcl_invitadosexcluidos
Not Null	nn_nomtabla_campo	nn_tm_ditto_distrito_codigo
Check	ck_nomtabla_campo	ck_tt_inv_invitados_sexo
Unique	uq_nomtabla_campo	uq_tm_jue_juegos_codigo

5.20 Para la nomenclatura de jobs, se deben seguir las siguientes reglas:

- El nombre de cada job debe ser lo más descriptivo posible, reflejando las características del proceso de negocio al que pertenece.
- La longitud máxima permitida para los jobs es de 45 caracteres.
- Se deberá anteponer el prefijo identificador de tres (03) caracteres: "JOB", para indicar que se trata de un job.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 14 de 26

Luego de agregar el prefijo JOB, se debe colocar el identificador, para indicar que tipo de trabajo que va a realizar.

IDENTIFICADOR	DESCRIPCIÓN
T-SQL	El job ejecuta un script T-SQL.
CMD_EXEC	El job ejecuta comandos del sistema operativo o programa ejecutable.
Power Shell	El job ejecuta un script Power Shell.
SSIS	El job ejecuta paquetes de integration services.
ANALYSISCOMMAND	El job ejecuta el comando de Analysis Services (XMLA).
ANALYSISQUERY	El Job ejecuta consulta de Analysis Service (MDX, DMX)
OTROS	El job ejecuta otros comandos.

Ejemplos:

TIPO	SINTAXIS	EJEMPLO
T-SQL	job_tsqI_procesonegocio	job_tsqI_puntos_ad
SSIS	job_ssis_procesonegocio	job_ssis_navidad_grupousuarios

- **IMPORTANTE:** Para los jobs que ejecutan procesos ETL, ya sean desarrollados por el equipo de Data Management (DM) o desplegados durante un pase a producción, la nomenclatura del nombre del job deberá seguir los lineamientos definidos por el equipo de DM.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 15 de 26

5.21 Para la nomenclatura de índices, se deben seguir las siguientes reglas:

- La longitud máxima permitida para los índices es de 30 caracteres.
- Se deberá anteponer el prefijo identificador de tres (03) caracteres: "IDX", para indicar que se trata de un índice.

Luego de agregar el prefijo IDX, se debe colocar el identificador, para indicar que tipo de índice.

IDENTIFICADOR	DESCRIPCIÓN
C	Índice Clúster.
CUV	Índice Clúster para vista.
D	Índice No Clúster duplicado.
U	Índice No Clúster único.
F	Índice No Clúster filtrado.
C	Índice No Clúster comprimido.
I	Índice No Clúster duplicado con Include.

Ejemplos:

TIPO	SINTAXIS	EJEMPLO
C	idx_c_nombretabla	idx_c_tt_operacion
D	idx_d_nombretabla_nro.	idx_d_tt_movimiento_01
U	idx_u_nombretabla_nro.	idx_u_tm_juegos_01
F	idx_f_nombretabla_nro.	idx_f_tt_grupojugadores_03
C	idx_c_nombretabla_nro.	idx_c_tm_distrito_02
I	idx_i_nombretabla_nro.	idx_i_tt_premios_01

5.22 La documentación de los objetos programados en bases de datos como Stored Procedures, Views, Functions, Triggers, etc., es fundamental para una adecuada gestión de cambios y mantenimiento de los sistemas. Por esta razón, se deben tener en cuenta las siguientes consideraciones:

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 16 de 26

5.23 Para la **documentación de bloque de código** cada objeto programado debe estar documentado con la siguiente información:

/*-----

Nombre: Nombre del procedimiento, función, vista, trigger u otro objeto correspondiente.

Descripción: Breve explicación del propósito y funcionalidad del objeto.

Autor: Nombres y apellidos del desarrollador responsable de su creación.

Fecha de creación: Fecha en la que fue creado el objeto.

Proyecto: Nombre del proyecto al que pertenece el objeto programado.

Versión: Número o identificador de versión correspondiente al proyecto o componente.

Parámetros: Detalle de los parámetros que utiliza el objeto.

Servidor y BD: Nombre de la instancia del servidor y de la base de datos donde se encuentra implementado el objeto.

-----*/

Ejemplo:

```

1  /*-----
2
3  Nombre: [sch_survey].[usp_get_surveyviews_by_id_jugador_online]
4
5  Descripción: Obtener la fecha de visualización y el interceptor de la encuesta por jugador y en un rango de fecha
6
7  Autor: Carlos Castro de la Cruz
8
9  Fecha de creación: 04/12/2024
10
11 Proyecto: Encuesta QUALTRICS.
12
13 Versión: 1.0
14
15 Parámetros: [sch_survey].[usp_get_surveyviews_by_id_jugador_online] '20345'
16
17 Instancia - BD - Esquema: PRODDB01 - [db_api_invitados_casino_online] - [sch_survey]
18
19 -----*/
20
21 A continuación de la cabecera general, se deberá incluir la cabecera del control de cambios:
22
23 {
24
25 @Nro. Fecha Autor Descripción
26
27 }
```




LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS

Código: GTI-OT016

Versión: 01

Página: 17 de 26

```
1  @1 06/12/24      ccastro Habilitar el estado del jugador para sorteo.
2
3  WHERE A.estado = 1 - @1
4
5  AND B.estado = 1 - @1
6
7  AND C.estado = 1
8
9  ORDER BY B.id_etapa asc
```

5.24 Para la **documentación para diccionario de datos**, es fundamental incluir comentarios durante la creación física de la base de datos, especialmente cuando se realizan cambios en la estructura directamente sobre la base de datos física. Esta práctica permitirá mantener un registro claro y detallado de las modificaciones realizadas, facilitando su comprensión y seguimiento a lo largo del tiempo.

Posteriormente, será posible actualizar el modelo de datos y el diccionario de datos mediante procesos de ingeniería inversa, utilizando herramientas CASE de modelado. Esto asegura la coherencia entre el modelo lógico y la implementación física, además de mantener la documentación técnica actualizada.

Ejemplo para colocar una descripción a la tabla:

```
1 EXEC sp_addextendedproperty
2
3     @name = N'MS_Description',
4
5     @value = N'Tabla mae aplicacion',
6
7     @level0type = N'SHEMA',
8
9     @level0name = N'sch_gespago',
10
11    @level1type = N'TABLE',
12
13    @level1name = N'mae_aplicacion',
```

Ejemplo para colocar una descripción a una columna:

```
1 EXEC sp_addextendedproperty @name = N'MS_Description',
2
3     @value = N'Identificador único de la aplicación.',
4
5     @level0type = N'SHEMA',
6
7     @level0name = N'sch_gespago',
8
9     @level1type = N'TABLE',
10
11    @level1name = N'mae_aplicacion',
12
13    @level2type = N'COLUMN',
14
15    @level2name = N'id_aplicacion';
```

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 18 de 26

5.25 Para la **documentación para parámetros, variables y constante**, se deben seguir las siguientes reglas:

- El nombre de las variables, parámetros y constantes debe ser lo más representativo e intuitivo posible, reflejando claramente su propósito dentro del sistema o proceso que soportan. Siempre deben ser definidos en singular y con palabras simples que faciliten su interpretación.
- La longitud máxima permitida para los nombres de variables, parámetros y constantes es de 30 caracteres.
- Cuando el nombre esté compuesto por más de una palabra, se deberá utilizar el carácter de guión bajo (_) como separador entre palabras, para garantizar la legibilidad del identificador.

```
1 <in o out>_ <parámetro o variable> _<tipo_de_dato>_<nombre>
2
```

ELEMENTO	DETALLE
<in o out>	in = entrada
	out = salida
<parámetro o variable>	p = parámetro
	v = variable
<tipo de dato>	v = varchar
	n = number
	d = date
	dt = datetime
	c = char
	d = decimal
<nombre>	Debe estar en singular, simple, representativo e intuitivo.

Ejemplo:

```
1 in_p_c_codinvitado = Parámetro de entrada con tipo de datos char, y almacena código invitado.
2
3 out_p_n_puntos = Parámetro de salida con tipo de dato numérico, y almacena los puntos.
4
5 in_v_n_codgrpjuego = Variable de entrada con tipo de dato numérico y almacena código grupo de juego.
6
7 out_v_d_cumpleaños = Variable de salida con tipo de dato fecha y almacena la fecha de nacimiento.
8
9 out_v_n_nrodias = Variable de salida con tipo de dato numérico y almacena numero de días.
```

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 19 de 26

La nomenclatura de una constante, es decir, una variable cuyo valor permanece inalterable durante la ejecución de un programa, ya sea dentro de un procedimiento almacenado o una función, debe seguir una convención clara y estandarizada que facilite su identificación y comprensión.

SINTAXIS	EJEMPLO
<@>_<NombreMayuscula>	@CODJUEGO=101

LINEAMIENTOS PARA BASES DE DATOS NOSQL

5.26 Una base de datos NoSQL es un tipo de sistema de gestión de bases de datos que no utiliza el modelo relacional tradicional basado en tablas, filas y columnas. En lugar de eso, NoSQL ofrece modelos de datos más flexibles, diseñados para manejar grandes volúmenes de datos distribuidos, no estructurados o semiestructurados.

5.27 Las guías y buenas prácticas, son las siguientes:

- Es **OBLIGATORIO** asegurar que las tablas no tengan acceso público.
- Todas las tablas deben contar con un mecanismo de cifrado para proteger la información sensible almacenada.
- Se recomienda habilitar un tipo de réplica como parte de una estrategia de balanceo de carga y recuperación ante desastres.
- Activar el proceso de respaldo sobre las tablas, como una medida de contingencia y recuperación de datos en caso de errores o pérdidas.
- Se sugiere considerar la creación de índices secundarios (locales o globales) para mejorar las consultas sobre atributos que no forman parte de la clave primaria.
- Activar la opción de “Protección contra Eliminación” en las tablas para evitar eliminaciones accidentales o no autorizadas.
- Se recomienda seleccionar el modo de “Capacidad Bajo Demanda”, ya que simplifica la gestión y la facturación, cobrando solo por las operaciones de lectura y escritura efectivamente realizadas por la aplicación.

5.28 Para la nomenclatura de una **tabla**, se deben seguir las siguientes reglas:

- El nombre de una tabla deberá ser lo más descriptivo posible y utilizar el singular, procurando incluir las características más relevantes de la información que contendrá.
- El nombre de la tabla no debe exceder los 255 caracteres, debe estar libre de tildes y espacios en blanco, y, en el caso de nombres compuestos, estos deben estar separados por guión bajo (_).

IMPORTANTE: Se sugiere asignar etiquetas a las tablas desde su creación, ya que estas permiten organizar, clasificar y gestionar los recursos de forma más eficiente, facilitando tareas como monitoreo, control de costos y automatización.



LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS

Código: GTI-OT016

Versión: 01

Página: 20 de 26

1 <TipoJuego>-<NomPy>-<NomModulo>-<Nomtabla>

ELEMENTO	DETALLE
<TipoJuego>	caf = Casino Físico
	col = Casino Online
<NomPy>	Corresponde al nombre del proyecto. Técnicamente, representa el nombre de la base de datos.
<NomModulo>	Representa el nombre funcional del módulo o subsistema dentro del proyecto. Técnicamente, equivale al nombre del esquema.
<Nomtabla>	Es el nombre específico de la tabla que describe la información contenida. Debe ser descriptivo, en singular, sin tildes ni espacios, y utilizar guiones bajos (_) para separar palabras.

Ejemplos:

- col-cms_ext-tournament-features
- col-cms_ext-tournament-filters

5.29 Para la nomenclatura de un **atributo**, se deben seguir las siguientes reglas:

- El nombre de un atributo deberá ser lo más descriptivo posible y utilizar el singular, procurando incluir las características más relevantes de la información que contendrá.
- El nombre de la tabla no debe exceder los 65,536 caracteres, debe estar libre de tildes y espacios en blanco, y, en el caso de nombres compuestos, estos deben estar separados por guión bajo (_).
- Para los atributos tipo Partition Key y Sort Key no debe exceder los 255 caracteres.
- Los nombres de los atributos Partition Key y Sort Key se mantendrán siempre para cualquier tipo de tabla.

Donde:

ELEMENTO	DETALLE
<pk>	Indica que es atributo tipo Partition Key.
<sk>	Indica que es un atributo tipo Sort Key.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 21 de 26

- Para el resto de los atributos se usará la siguiente estructura:

9 <tp>_<nomatr>

Donde:

ELEMENTO	DETALLE
<tp>	Indica el tipo de dato: • Cadena (ca) • Número (nu) • Binario (bi))
<nomatr>	Indica el nombre del atributo.

Ejemplo:

- **Tabla:** col-cms_ext-tournament-features
- **Atributo:** ca_nombre
- **Atributo:** ca_compañía
- **Atributo:** nu_grupo

5.30 Para la nomenclatura de un **índice secundario**, se deben seguir las siguientes reglas:

- El nombre de un índice secundario deberá ser descriptivo y reflejar con claridad su propósito dentro del modelo de datos. Su nombre debe permitir identificar, de forma inmediata, la entidad sobre la cual se aplica y el atributo clave que se utilizará para las operaciones de consulta.

5.31 Para la nomenclatura de un **índice**, se debe seguir las siguientes reglas:

- La longitud máxima permitida para los índices es de 255 caracteres.
- Todos los caracteres deben estar en minúsculas.
- Cuando el nombre esté compuesto por más de una palabra, se deberá utilizar el carácter de guión bajo (_) como separador entre palabras, para garantizar la legibilidad del identificador.
- Se debe anteponer un prefijo que indique el tipo de índice, para facilitar su clasificación y mantenimiento. Existen dos tipos de índices:
 - **Índice Secundario Local (Local Secondary Index - LSI):** Utiliza la misma clave de partición (Partition Key) que la tabla principal, pero permite definir una clave de ordenación diferente (Sort Key).

```

1 gsi_<entidad>_<campoindexado>
2 Ejemplo:
3 gsi_col-cms_ext-tournament-features_compañía

```

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 22 de 26

- **índice Secundario Global (Global Secondary Index - GSI):** Los atributos que lo conforman son aquellos que están fuera de la clave principal (Primary Key), de la tabla original, es decir, te permite crear una nueva clave de partición (Partition key) y una nueva clave de ordenación (Sort Key).

```

1 lsi_<entidad>_<campoordenado>
2 Ejemplo:
3 lsi_col-cms_ext-tournament-features_grupo

```

LINEAMIENTOS PARA DISEÑO DE CLAVES EN BASES DE DATOS NOSQL

- 5.32 El diseño de la **clave de partición** es un aspecto fundamental en la estructuración de tablas en bases de datos NoSQL, ya que determina como se almacenan y acceden los datos. La clave de partición debe seleccionarse cuidadosamente de acuerdo con los requerimientos del negocio, considerando que será el campo principal utilizado para búsquedas, conteos y segmentación de datos. Toda la estructura y organización interna de la tabla se basa en esta clave, ya que este tipo de bases de datos agrupa los registros (ítems) según su valor.

SINTAXIS	EJEMPLO
"<CLAVE> <identificador único>"	IDJUGADOR 12345678

Los beneficios son los siguientes:

- Distribución uniforme de los datos, evitando puntos calientes.
 - Consultas eficientes sin necesidad de operaciones de tipo “**Scan**”.
 - Evitar claves con valores repetitivos para reducir colisiones y congestiones.
- 5.33 El diseño de la **clave de ordenación** es crucial para estructurar los datos dentro de una partición de manera eficiente, permitiendo consultas más rápidas y organizadas. Esta clave complementa a la clave de partición y se utiliza para ordenar y categorizar los registros dentro de una misma partición.

SINTAXIS	EJEMPLO
<CLAVE1> <valor1> <CLAVE2> <valor2> <CLAVE3> <valor3>	MAQUINA Maq- 1231 CUPON 10DESC PUNTOS 302

Los beneficios son los siguientes:

- Permite consultas jerárquicas dentro de una misma clave de partición, organizando los registros de manera estructurada.
- Facilita búsquedas eficientes por prefijo (begins_with), optimizando la búsqueda de datos con patrones comunes.
- Optimiza la organización y recuperación de datos estructurados, mejorando la eficiencia en consultas relacionadas con atributos específicos.

	LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS	Código: GTI-OT016
		Versión: 01
		Página: 23 de 26

LINEAMIENTOS PARA CONSULTAS EN BASES DE DATOS NOSQL

- 5.34 El uso de la operación Scan **NO ESTÁ PERMITIDO** en entornos productivos, ya que recorre toda la tabla y puede generar un alto consumo de recursos, lo que impacta negativamente en el rendimiento y la escalabilidad de la base de datos. Se debe evitar siempre que sea posible. En su lugar, se debe utilizar Query con clave de partición (Partition Key) y clave de ordenación (Sort Key) definidas.

Ejemplo de uso incorrecto:

- Uso **incorrecto** de Scan en Python

```
1 table = dynamodb.Table("Jugadores")
2
3 response = table.scan()
4 print(response["Items"])
```

- Uso **incorrecto** de Scan en C#

```
1 var request = new ScanRequest
2 {
3     TableName = "Jugadores"
4 };
5
6 var response = await client.ScanAsync(request);
7 Console.WriteLine(response.Items);
```

- Uso **incorrecto** de Scan en Node.js

```
1 const command = new ScanCommand({ TableName: "Jugadores" });
2
3 const response = await docClient.send(command);
4 console.log(response.Items);
```

Problema: Esta consulta escanea todos los registros, lo que afecta el rendimiento y los costos.

Ejemplo de uso correcto:

- Uso **correcto** de Query en Python

```
1 response = dynamodb.query(
2     TableName="Jugadores",
3     KeyConditionExpression="PK = :pk",
4     ExpressionAttributeValues={":pk": {"S": "IDJUGADOR|12345678"}}
5 )
6
7 print(response["Items"])
```



LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS

Código: GTI-OT016

Versión: 01

Página: 24 de 26

- **Uso correcto de Query en C#**

```
1 var request = new QueryRequest
2 {
3     TableName = "Jugadores",
4     KeyConditionExpression = "PK = :pk",
5     ExpressionAttributeValues = new Dictionary<string, AttributeValue>
6     {
7         { ":pk", new AttributeValue { S = "IDJUGADOR|12345678" } }
8     }
9 };
10
11 var response = await client.QueryAsync(request);
12 Console.WriteLine(response.Items);
```

- **Uso correcto de Query en Node.js**

```
1 const command = new QueryCommand({
2     TableName: "Jugadores",
3     KeyConditionExpression: "PK = :pk",
4     ExpressionAttributeValues: { ":pk": "IDJUGADOR|12345678" }
5 });
6
7 const response = await DynamoDBDocumentClient.from(client).send(command);
8 return response.Items;
```

Ventaja: Recupera solo los registros necesarios sin afectar la base de datos.

LINEAMIENTOS PARA USO DE PREFIJOS EN SK PARA FILTRADO EFICIENTE EN BASES DE DATOS NOSQL

5.35 Para realizar consultas sobre el primer parámetro de la clave de ordenación (Sort Key), se recomienda utilizar la función `begins_with()`, ya que permite filtrar eficientemente los registros cuyo valor en la clave inicia con un prefijo específico. En caso de requerir búsquedas sobre otros segmentos o partes internas del valor, puede emplearse la función `contains()`, aunque esta opción implica un mayor consumo de recursos y menor eficiencia en comparación con `begins_with()`.

Ejemplos:

- **Obtener datos del primer parámetro en Python**

```
1 response = dynamodb.query(
2     TableName="Jugadores",
3     KeyConditionExpression="PK = :pk AND begins_with(SK, :sk_prefix)",
4     ExpressionAttributeValues={
5         ":pk": {"S": "IDJUGADOR|12345678"},
6         ":sk_prefix": {"S": "MAQUINA|Maq-1231"}
7     }
8 )
9
10 print(response["Items"])
```




LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS

Código: GTI-OT016

Versión: 01

Página: 25 de 26

- Obtener **datos del primer parámetro** en C#

```
1 var request = new QueryRequest
2 {
3     TableName = "Jugadores",
4     KeyConditionExpression = "PK = :pk AND begins_with(SK, :sk_prefix)",
5     ExpressionAttributeValues = new Dictionary<string, AttributeValue>
6     {
7         { ":pk", new AttributeValue { S = "IDJUGADOR|12345678" } },
8         { ":sk_prefix", new AttributeValue { S = "MAQUINA|Maq-1231" } }
9     }
10 };
11
12 var response = await client.QueryAsync(request);
13 Console.WriteLine(response.Items);
```

- Obtener **datos del primer parámetro** en Node.js

```
1 const command = new QueryCommand({
2     TableName: "Jugadores",
3     KeyConditionExpression: "PK = :pk AND begins_with(SK, :sk_prefix)",
4     ExpressionAttributeValues: {
5         ":pk": "IDJUGADOR|12345678",
6         ":sk_prefix": "MAQUINA|Maq-1231"
7     }
8 });
9
10 const response = await DynamoDBDocumentClient.from(client).send(command);
11 return response.Items;
```

- Obtener **datos de cualquier parámetro** en Python

En este caso buscaremos <CUPON> que está en el segundo orden para el siguiente registro: "MAQUINA|Maq-1231|CUPON|10DESC|PUNTOS|302"

```
1 response = dynamodb.query(
2     TableName="Jugadores",
3     KeyConditionExpression="PK = :pk",
4     FilterExpression="contains(SK, :cupon)",
5     ExpressionAttributeValues={
6         ":pk": {"S": "IDJUGADOR|12345678"},
7         ":cupon": {"S": "CUPON|10DESC"}
8     }
9 )
10
11 print(response["Items"])
```

- Obtener **datos de cualquier parámetro** en C#

En este caso buscaremos <CUPON> que está en el segundo orden para el siguiente registro: "MAQUINA|Maq-1231|CUPON|10DESC|PUNTOS|302"



LINEAMIENTOS DE DEFINICION DE ESTANDARES Y BASE DE DATOS

Código: GTI-OT016

Versión: 01

Página: 26 de 26

```
1 var request = new QueryRequest
2 {
3     TableName = "Jugadores",
4     KeyConditionExpression = "PK = :pk",
5     FilterExpression = "contains(SK, :cupon)",
6     ExpressionAttributeValues = new Dictionary<string, AttributeValue>
7     {
8         { ":pk", new AttributeValue { S = "IDJUGADOR|12345678" } },
9         { ":cupon", new AttributeValue { S = "CUPON|10DESC" } }
10    }
11 };
12
13 var response = await client.QueryAsync(request);
14 Console.WriteLine(response.Items);
```

- Obtener **datos de cualquier parámetro** en Node.js

En este caso buscaremos <CUPON> que está en el segundo orden para el siguiente registro: "MAQUINA|Maq-1231|CUPON|10DESC|PUNTOS|302"

```
1 const command = new QueryCommand({
2     TableName: "Jugadores",
3     KeyConditionExpression: "PK = :pk",
4     FilterExpression: "contains(SK, :cupon)",
5     ExpressionAttributeValues: {
6         ":pk": "IDJUGADOR|12345678",
7         ":cupon": "CUPON|10DESC"
8     }
9 });
10
11 const response = await DynamoDBDocumentClient.from(client).send(command);
12 return response.Items;
```

6. REGISTROS:

No aplica.

7. ANEXOS:

No aplica.

8. CONTROL DE CAMBIOS:

No aplica.